



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3133 – HIGH PERFORMANCE DATA PROCESSING SECTION 2

PROJECT

**OPTIMIZING HIGH-PERFORMANCE DATA PROCESSING FOR LARGE-SCALE
WEB CRAWLERS**

LECTURER: DR. SEAH CHOON SEAN

SUBMISSION DATE:

24 JUNE 2026

STUDENT NAME	MATRIC NO
CHERYL CHEONG KAH VOON	A23CS0060
CHAU YING JIA	A23CS0213
LAU YEE WEN	A23CS0099
POH LOK YEE	A23CS0262

Table of Contents

1.0	Introduction	4
1.1	Project Background.....	4
1.2	Objectives	4
1.3	Target website and data to be extracted	5
2.0	System Design & Architecture	5
2.1	System architecture.....	6
2.2	Tools and frameworks used.....	7
2.3	Roles of team members	7
3.0	Data Collection.....	8
3.1	Crawling method (pagination, rate-limiting, async).....	8
3.2	Number of records collected.....	9
3.3	Ethical considerations	9
4.0	Data Processing	10
4.1	Data Cleaning Process.....	10
4.2	Data Transformation Process.....	11
4.3	Data structure.....	11
5.0	Optimization Techniques.....	12
5.1	Overview of Optimization Techniques	13
5.2	Pandas Optimized Pipeline.....	13
5.3	Polars Lazy Execution Pipeline.....	14
5.4	DuckDB SQL-Based Processing	15
6.0	Performance Evaluation	16
6.1	Before vs after optimization	16
6.2	Time, memory, CPU usage, throughput.....	17
6.3	Visualization.....	18
7.0	Challenges & Limitations	21
7.1	Challenges.....	21
7.2	Limitations of the solution	22
8.0	Conclusion & Future Work.....	22

8.1	Summary of findings.....	22
8.2	Improvements	23
9.0	References	24
10.0	Appendices.....	24
	Screenshots of Output.....	24
	Links to full code repo or dataset.....	25

1.0 Introduction

1.1 Project Background

In the contemporary digital landscape, the exponential growth of big data has made the ability to efficiently collect and transform raw online information into actionable insights a highly valuable capability. This project specifically focuses on the application of High-Performance Computing (HPC) methodologies to optimize web-scale data collection, making the entire process significantly faster and more resource-efficient.

Currently, Malaysia's online employment market is expanding rapidly, creating a strong demand for advanced analytical tools capable of tracking job vacancies and hiring trends. Although public platforms like Jobstreet.com host a wealth of career opportunities, relying on manual data collection across thousands of listings is incredibly time-consuming and highly impractical. To overcome this limitation, an automated web scraper has been engineered to systematically extract job details, thereby ensuring a highly reliable, rapid, and accurate data collection workflow.

The core technical scope of this work involves developing practical expertise in parallel processing paradigms, with a specific focus on implementing multithreading, multiprocessing, and distributed computing architectures. By deploying these advanced techniques to process extensive datasets from Jobstreet.com, this project aims to demonstrate how large-scale platforms can be analyzed efficiently to generate meaningful, data-driven insights for both job seekers and employers.

1.2 Objectives

The primary objective of this technical initiative is to build, optimize, and benchmark advanced data processing frameworks that significantly accelerate data ingestion. By establishing a high-performance architecture, the project aims to optimize the web scraper so that it can seamlessly handle high volumes of streaming data without encountering performance bottlenecks or system crashes. To achieve this, the specific milestones of the project include:

- **High-Volume Data Ingestion:** Designing and deploying an optimized web crawler capable of successfully extracting a baseline dataset of at least 100,000 structured records from Jobstreet.com.
- **Automated Pipeline Refinement:** Establishing an automated data pipeline to thoroughly clean, transform, and structuralize raw data fields by removing duplicates and standardizing information to ensure absolute data integrity.
- **HPC Engine Acceleration:** Implementing at least three distinct high-performance optimization techniques—specifically using multithreading for concurrent tasks, multiprocessing to utilize multiple CPU cores, and asynchronous programming with semaphores to compress computational cycle times.

- **Rigorous Performance Evaluation:** Conducting clear before-and-after testing to evaluate the system pre- and post-optimization, thereby formally proving system speedups and efficiency gains.
- **Technical Documentation and Presentation:** Constructing a professional final technical report alongside visualization charts and slides to effectively explain the architectural design, system performance telemetry, and data discoveries

1.3 Target website and data to be extracted

The target platform chosen for this data engineering pipeline is Jobstreet.com, which stands as one of the leading digital recruitment networks in Malaysia. The scraping mechanism is programmed to target specific structural parameters within individual job advertisements, capturing essential data points such as job titles, company profiles, geographical locations, and offered salary ranges.

To analyse the efficiency and scalability of the HPC implementation, the pipeline's execution will be continuously monitored across several core technical categories:

- **System Resource Telemetry:** Monitoring the exact CPU utilization percentage and volatile memory consumption (in MB) during active processing cycles.
- **Data Throughput Speeds:** Measuring the total execution time in seconds alongside the number of rows successfully processed per second.
- **Data Scale Metrics:** Tracking the total aggregated rows and completed data sub-blocks successfully ingested by the system.
- **Architectural Optimization Tiers:** Profiling and comparing the performance differences across different processing setups, specifically Multithreading, Multiprocessing, and Distributed Computing.

By thoroughly analyzing these high-performance computing metrics, it is possible to evaluate how effectively the scraping architecture handles heavy data workloads. The resulting structured dataset will provide a clear overview of the current Malaysian job market while ensuring that the technical, legal, and ethical responsibilities of automated data extraction are maintained.

2.0 System Design & Architecture

This section presents the overall architecture of the proposed system for collecting and processing JobStreet Malaysia job listings. It describes the workflow of the data processing pipeline, the tools and frameworks used, and the responsibilities assigned to each team member throughout the project.

2.1 System architecture

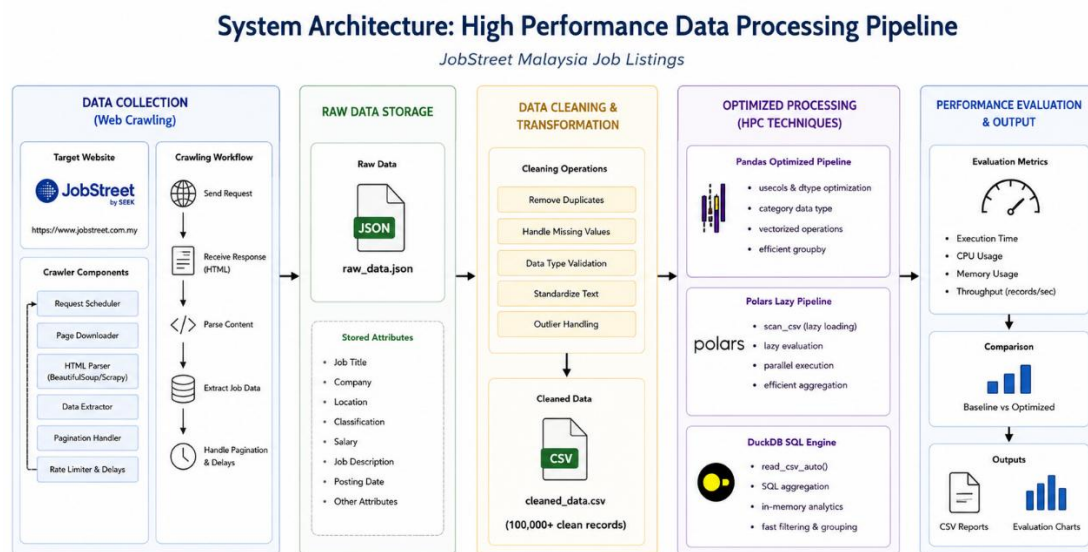


Figure 2.1: System Architecture Diagram

Figure 2.1 illustrates the overall architecture of the JobStreet Malaysia High-Performance Data Processing Pipeline. The architecture consists of five major layers: Data Collection, Raw Data Storage, Data Cleaning and Transformation, Optimized Processing, and Performance Evaluation. These layers work together to support the collection, processing, optimization, and evaluation of large-scale job listing data.

The process begins with the **Data Collection** layer, where job listing information is extracted from JobStreet Malaysia using an automated web crawler. The crawler sends requests to JobStreet webpages, retrieves the HTML content, parses the responses, and extracts relevant job attributes such as job title, company, location, classification, and salary. Pagination handling and request delays are also implemented to ensure efficient and responsible data collection.

The extracted records are then stored in the **Raw Data Storage** layer in JSON format. This layer preserves the original scraped data before any preprocessing activities are performed. The raw dataset serves as the primary source for subsequent cleaning and transformation processes.

Next, the **Data Cleaning and Transformation** layer improves the quality and consistency of the collected data. Various preprocessing operations are applied, including duplicate removal, missing value handling, data type validation, and text standardization. After the cleaning process is completed, the dataset is stored in CSV format for further analysis and processing.

The cleaned dataset is then passed to the **Optimized Processing** layer, where high-performance processing techniques are applied using Pandas, Polars, and DuckDB. These frameworks improve processing efficiency through memory optimization, vectorized operations, lazy evaluation, parallel execution, and SQL-based analytical processing. The objective of this layer is to reduce processing overhead while maintaining data accuracy.

Finally, the **Performance Evaluation** layer measures the effectiveness of the implemented optimization techniques. Several performance metrics, including execution time, CPU usage, memory consumption, and throughput, are collected and compared. The evaluation results are presented through tables and visualizations to identify the most efficient processing approach for handling large-scale job listing data.

2.2 Tools and frameworks used

Table 2.1 summarizes the tools and frameworks used throughout the project.

Table 2.1 Tools and Frameworks

Tool / Framework	Purpose
Python	Primary programming language used for web scraping, data processing, optimization, and performance evaluation
Playwright	Browser automation framework used to retrieve dynamic web content from JobStreet Malaysia
BeautifulSoup	HTML parsing library used to extract job information from webpage content
Asyncio	Supports asynchronous crawling and concurrent execution to improve crawler performance
JSON	Stores raw scraped job data before preprocessing
Pandas	Data cleaning, transformation, aggregation, and structured data processing
Polars	High-performance DataFrame library supporting lazy execution and parallel processing
DuckDB	SQL-based analytical engine used to process CSV data efficiently without fully loading it into memory
Matplotlib	Generates performance comparison charts and visualizations
Google Colab	Cloud-based development environment used for implementation and testing
Google Drive	Stores datasets and project files for collaboration and accessibility

2.3 Roles of team members

As shown in Table 2.2, the project was divided into four main components to ensure an efficient workflow and balanced workload distribution among team members. Each member was assigned specific responsibilities based on the project requirements, including web crawling, data cleaning, optimization, and performance evaluation.

Table 2.2 Team roles

Team Members	Responsibilities
--------------	------------------

Poh Lok Yee	Developed the web crawler for JobStreet Malaysia, implemented data extraction logic, handled pagination and asynchronous crawling, and stored the collected data in raw JSON format.
Chau Ying Jia	Developed the data cleaning pipeline, performed data preprocessing, removed duplicate records, handled missing values, standardized data formats, and generated the cleaned dataset in CSV format.
Lau Yee Wen	Designed the system architecture, implemented optimization techniques using Pandas, Polars, and DuckDB, and developed the optimization pipeline for efficient large-scale data processing.
Cheryl Cheong Kah Voon	Conducted performance evaluation by comparing baseline and optimized implementations, measured execution time, CPU usage, memory consumption, throughput, and generated performance analysis charts and reports.

3.0 Data Collection

3.1 Crawling method (pagination, rate-limiting, async)

The data collection process was implemented using an automated web scraping pipeline built with Playwright and asynchronous programming (asyncio). The crawler targets job listings from JobStreet Malaysia, systematically extracting data across multiple job classifications and pages.

Pagination Strategy:

The crawler navigates through job listings using URL-based pagination. Each classification category (1–30) is iterated sequentially, and for each category, multiple pages are accessed using the following structure:

`https://my.jobstreet.com/jobs?classification={id}&page={page}`

The system continues paginating until either no job listings are returned or the maximum page limit is reached.

Asynchronous Execution:

To improve efficiency, the crawler uses asynchronous concurrency with a semaphore-controlled limit of 8 parallel browser sessions. Each classification is assigned a worker, allowing multiple categories to be scraped simultaneously while preventing system overload.

HTML Parsing:

After Playwright loads each webpage, the rendered HTML content is passed to BeautifulSoup for parsing. BeautifulSoup locates each job listing and extracts structured information such as the job title, company name, location, classification, and salary using the webpage's HTML elements and attributes. This separates browser automation from data extraction, making the scraping pipeline more efficient and easier to maintain.

Rate Limiting Strategy:

To reduce the risk of detection and avoid overwhelming the server, controlled delays are introduced at multiple levels:

- Random delay between page loads (2–4 seconds)
- Additional delay between page requests (1.5–3.5 seconds)
- Retry mechanism with exponential backoff (up to 3 attempts per request)

These measures help simulate human-like browsing behavior and improve scraping stability.

3.2 Number of records collected

The dataset was collected across 30 job classification categories from JobStreet Malaysia. Each classification represents a different job sector allowing for a diverse and balanced representation of the job market.

The crawler iterates through all classifications and extracts job listings from each category until either the page limit is reached or no further records are available.

Total Job Classifications: 30 categories (Classification ID 1–30)

Target Records: 105094 job listings

Execution Time: 64.3 minutes

Data Granularity: Individual job postings per classification

Output Format: JSON Lines (JSONL)

Each record includes structured fields such as job title, company name, location, classification, and salary information. The inclusion of classification data ensures that each job listing is traceable to its respective industry category, enabling category-level analysis such as job distribution and demand trends across sectors.

The final dataset size is recorded upon completion of the crawling process and stored in `data/raw_datas.json`.

3.3 Ethical considerations

This data collection process follows several ethical scraping practices to ensure responsible usage of publicly available information.

- **Respect for Public Data Access:** Only publicly accessible job listings were collected. No login-protected or private user data was accessed.

- **Minimizing Server Load:** The crawler incorporates rate limiting, randomized delays, and concurrency control to prevent excessive requests to JobStreet's servers, reducing potential impact on system performance.
- **Non-Disruptive Behavior:** The scraper does not perform actions that modify, submit, or interact with the website beyond normal page loading. It strictly operates in a read-only manner.
- **Data Usage Purpose:** The collected data is intended solely for academic and analytical purposes, such as data engineering practice and job market analysis. It is not used for commercial redistribution.

4.0 Data Processing

After the raw data was collected by the crawler and saved as `raw_data.json` from JobStreet Malaysia, the next phase was to clean and transform it into a structured, analysis-ready format. This task was handled using a Jupyter Notebook named `clean_data.ipynb`. Based on Figure 4.1, the entire pipeline was written in Python using mainly these three libraries: `json` for reading the raw file, `re` for regex-based text extraction and `pandas` used for all data manipulation. The final cleaned dataset was exported as a single CSV file for further analysis.

4.1 Data Cleaning Process

The raw dataset initially contained 105,094 records across 5 columns (`job_title`, `company`, `location`, `classification`, and `salary`). Once the JSONL file was loaded into a Pandas DataFrame, several cleaning operations were applied to prepare the data:

4.1.1 Duplicate removal

This process was performed using the `drop_duplicates()` function to eliminate redundant records that were captured multiple times during the automated web crawling phase. A total of 79,006 exact duplicate rows were removed, which left 26,088 unique records for further processing.

4.1.2 Missing value handling

This process was handled based on the importance of the columns. The `job_title`, `company`, `location` and `classification` fields were identified as required data. Rows missing any of these required fields (14 rows in total) were dropped entirely to maintain data integrity. For the optional salary column, empty values were filled with the placeholder text "Not Disclosed" to preserve row completeness. After this step, 26,074 rows remained.

4.2 Data Transformation Process

Following the initial cleaning stage, several transformation steps were carried out to standardize all data fields for the final analysis.

4.2.1 Whitespace normalization

This process was applied to all string columns. Multiple consecutive spaces were collapsed into a single space and leading or trailing whitespaces at the edges of the text were trimmed to ensure clean and uniform text data.

4.2.2 Classification formatting

The job classification text often contained wrapping parentheses such as "(Accounting)". These parentheses were stripped away from the text to provide clean, readable and consistent category names.

4.2.3 Salary data extraction and formatting Some salary strings contained non-breaking spaces (e.g., \xa0), which were first normalized into regular spaces. A regular expression (regex) function was then applied to extract the minimum and maximum numeric values from the text strings. For example, extracting the numbers from "RM 4,000 – RM 5,000 per month". These extracted numbers were stored in two newly created columns, salary_min and salary_max. Both new columns were converted to numeric float types to allow for future mathematical calculations.

4.2.4 Job title text processing

Job titles were first standardized by replacing multiple consecutive spaces with a single space and stripping any leading or trailing whitespace. Subsequently, the dataset was filtered to remove any listings containing non-ASCII characters, such as Chinese text, within the job title. This step ensured that only uniformly formatted, ASCII-compliant job titles were retained in the dataset for further processing.

4.2.5 Location standardization

Inconsistent location names and known abbreviations were mapped to standardized terms. For instance, variations like "Kl", "KL", "W.P. Kuala Lumpur" and "Wilayah Persekutuan Kuala Lumpur" were all replaced with a uniform "Kuala Lumpur". Similarly, "PJ" was standardized to "Petaling Jaya".

4.2.6 Data type validation and arrangement All text columns were strictly enforced and converted to string data types. Finally, the columns were rearranged into a logical order to make the dataset programmatically accessible and easier to read.

4.3 Data structure

The cleaned dataset is saved as cleaned_data.csv. It consists of 25209 data rows and 7 columns. Table 4.1 shows the structure of the cleaned dataset. Conceptually, the dataset holds categorical

and descriptive information such as the specific job, hiring company, the location, industry classification and the originally listed raw salary text. To support quantitative analysis, the structure has change to include numeric fields for the minimum and maximum monthly salary values, ensuring the text descriptions are isolated from the numbers so the dataset is strictly standardized and ready for further calculations or visualization.

Table 4.1 Data Structure of Cleaned_data.csv

Column Name	Data Type	Description & Example
job_title	String	The title or position of the job role. (e.g., "Senior Electrical Engineer- EHV Cables", "Accountant")
company	String	The name of the hiring organization or recruiter. (e.g., "AECOM", "Accenture Malaysia")
location	String	The city, state or general location where the job is based. (e.g., "George Town", "Johor Bahru")
classification	String	The industry or departmental category the job falls under. (e.g., "Real Estate & Property", "Information & Communication Technology")
salary	String	The raw salary text as originally listed. Often contains ranges or text if missing. (e.g., "RM 2,400 – RM 3,400 per month", "Not Disclosed")
salary_min	Numeric (Float)	The extracted minimum salary value from the raw text. Contains null if not disclosed. (e.g., 2400.0, null)
salary_max	Numeric (Float)	The extracted maximum salary value from the raw text. Contains null if not disclosed. (e.g., 3400.0, null)

5.0 Optimization Techniques

This section presents the optimization techniques applied to improve the efficiency of processing the JobStreet Malaysia dataset. Since the dataset contains a large number of job records, efficient data processing methods are required to reduce execution overhead and improve scalability. Three optimization approaches were implemented, namely the Pandas

Optimized Pipeline, Polars Lazy Execution Pipeline, and DuckDB SQL-Based Processing. These approaches were selected because they provide different optimization strategies for handling large-scale structured data efficiently.

5.1 Overview of Optimization Techniques

Table 5.1 summarizes the optimization techniques implemented in this project.

Table 5.1 Optimization Techniques

Framework	Optimization Technique
Pandas	Memory optimization, vectorized operations, efficient grouping
Polars	Lazy evaluation, query optimization, parallel execution
DuckDB	SQL-based analytical processing, direct CSV querying, optimized aggregation

These frameworks were evaluated to determine their effectiveness in processing large-scale job listing datasets.

5.2 Pandas Optimized Pipeline

The Pandas optimized pipeline reduces memory consumption and improves processing efficiency during data loading and transformation. Several optimization strategies were implemented using Pandas:

- `usecols` to load only the required columns
- `dtype` specification to reduce type inference overhead
- `.astype('category')` to optimize memory usage for repeated values
- Vectorized string operations using `.str.strip()` and `.str.title()`
- Boolean masking for efficient filtering
- `groupby(observed=True)` for optimized aggregation

These optimizations were implemented in the Pandas optimized pipeline as shown in Figure 5.1.

```

df_pandas = pd.read_csv(
    CSV_FILE,
    usecols=required_cols,
    dtype={
        "job_title": "string",
        "company": "string",
        "location": "string",
        "classification": "string",
        "salary": "string"
    },
    engine="c"
)

df_pandas["location"] = df_pandas["location"].astype("category")
df_pandas["classification"] = df_pandas["classification"].astype("category")

df_pandas["job_title"] = df_pandas["job_title"].str.strip().str.title()

mask_it = df_pandas["classification"].astype(str).str.contains(
    "Information & Communication Technology",
    case=False,
    na=False
)

pandas_summary = (
    df_pandas
    .groupby("classification", observed=True)
    .agg(total_jobs=("job_title", "count"))
)

```

Figure 5.1 Code Snippet Pandas

5.3 Polars Lazy Execution Pipeline

The Polars pipeline is designed to improve performance through lazy evaluation and parallel execution. Polars optimizes queries before execution, allowing faster computation and better resource utilization. The optimization strategies applied using Polars were:

- `scan_csv()` for lazy loading to avoid immediate computation
- Expression API for optimized column-based processing
- `.filter()` for efficient vectorized filtering
- `.group_by().agg()` for parallel aggregation operations
- Lazy execution using `.collect()` only at the final stage

These optimizations were implemented in the Polars pipeline as shown in Figure 5.2.

```

lf = pl.scan_csv(CSV_FILE)

lf_cleaned = lf.select([
    pl.col("job_title").str.strip_chars().str.to_titlecase(),
    pl.col("company").str.strip_chars().str.to_titlecase(),
    pl.col("classification").str.strip_chars()
])

lf_it_jobs = lf_cleaned.filter(
    pl.col("classification").str.contains(
        "Information & Communication Technology",
        literal=True
    )
)

polars_summary = (
    lf_cleaned
    .group_by("classification")
    .agg([
        pl.count().alias("total_jobs"),
        pl.col("company").n_unique().alias("total_companies")
    ])
    .collect()
)

```

Figure 5.2 Code Snippet Polars

5.4 DuckDB SQL-Based Processing

The DuckDB pipeline utilizes an in-process analytical SQL engine to perform high-performance processing directly on CSV files. By executing optimized SQL queries without loading the entire dataset into memory, DuckDB is suitable for large-scale and aggregation-intensive workloads. The optimization strategies applied include:

- `read_csv_auto()` for direct CSV querying
- SQL-based filtering and aggregation
- GROUP BY optimization for analytical processing
- COUNT() and COUNT(DISTINCT) analytical functions
- Efficient query execution on large datasets

These optimizations were implemented in the DuckDB pipeline as shown in Figure 5.3.

```
duckdb_summary = con.execute(f"""
SELECT
    classification,
    COUNT(*) AS total_jobs,
    COUNT(DISTINCT company) AS total_companies
FROM read_csv_auto('{CSV_FILE}')
GROUP BY classification
ORDER BY total_jobs DESC
""").df()

duckdb_summary.head(10)
```

Figure 5.3 Code Snippet DuckDB

6.0 Performance Evaluation

6.1 Before vs after optimization

The main goal of this performance evaluation is to quantitatively evaluate the effectiveness of the optimization technology implemented by the whole project. The evaluation compared the baseline implementation with three optimized processing frameworks using the JobStreet datasets that containing 14105 jobs listing. The benchmark test process is divided into two main stages which is before optimization and after optimization.

Before Optimization

The baseline implementation is developed using the standard Pandas operations. Data loading and aggregation tasks is performed sequentially using the default CSV parser without memory optimization or data type compression technology. This implementation is a reference point for evaluation subsequent improvements.

After Optimization

There are three optimized processing pipelines were implemented and evaluated which are Optimized Pandas, Polar Lazy and DuckDB SQL.

The first is Optimized Pandas, which improves the baseline implementation by using usecols and memory-efficient classified data type to utilize selective column loading. This optimization reduces unnecessary memory allocation and improves processing efficiency.

Second, Polars employ a rust-based execution engine with native multi-threading support. Through lazy evaluation, query optimization and parallel execution, Polar aims to maximize computing efficiency and throughput.

Then, DuckDB provides an analytical database engine optimized for online analytical processing (OLAP) workloads. It uses SQL queries to perform in-memory vectorization operations and is widely recognized for its high-performance analysis ability.

6.2 Time, memory, CPU usage, throughput

To evaluate the performance of each processing framework, four benchmark indicators are collected and analysed:

- **Execution Time (seconds):** the exact duration required for the framework to complete the core data loading and aggregation operation. Smaller execution time indicates higher computing efficiency.
- **Memory Usage (MB):** The peak resident set size delta measures in Megabytes (MB) during operation. This indicator reflects the amount of memory resources allocated by the operating system when processing the dataset.
- **CPU Usage (%):** The percentage of processor resources used during operation. Higher CPU utilization usually indicates more efficient use of available computing resources.
- **Throughput:** The number of dataset rows processed per second. Higher throughput indicates stronger data processing capacity and better overall performance.

To improve measurement reliability, each framework was executed three times under identical experimented conditions. The average value of the three benchmark runs was used as the final reported results. The benchmark results for these performance metrics presented in the following tables.

Table 6.1 Comparison for Execution Time (seconds)

Method	Run 1	Run 2	Run 3	Average
Baseline Pandas	0.52	0.30	0.10	0.31
Optimized Pandas	0.18	0.16	0.11	0.15
Polars Lazy Execution	0.08	0.05	0.07	0.07
DuckDB SQL	0.20	0.22	0.29	0.23

Table 6.2 Comparison for Memory Usage (MB)

Method	Run 1	Run 2	Run 3	Average
Baseline Pandas	11.77	0	0	3.92
Optimized Pandas	0.003	0	0	0.001
Polars Lazy Execution	1.31	0.88	0.18	0.79
DuckDB SQL	0	0.04	0.02	0.02

Table 6.3 Comparison for CPU Usage (%)

Method	Run 1	Run 2	Run 3	Average
Baseline Pandas	15.35	23.35	45.20	27.99
Optimized Pandas	41.4	44.65	46.25	44.1
Polars Lazy Execution	12.6	30.9	7.15	16.88
DuckDB SQL	47.9	36.75	32.55	39.07

Table 6.4 Comparison for Throughput

Method	Run 1	Run 2	Run 3	Average
Baseline Pandas	48347.33	84165.99	253702.84	128736.72
Optimized Pandas	139392.47	161062.84	233893.68	178116.33
Polars Lazy Execution	318484.21	522422.87	361347.35	400751.48
DuckDB SQL	127504.29	116155.19	87175.65	110278.38

6.3 Visualization

Execution Time Comparison

In the figure 6.1 shows the execution time comparison between the four processing frameworks. Polars Lazy Execution achieved the shortest running time, followed by Optimized Pandas, DuckDB SQL and Baseline Pandas. The excellent performance of Polars Lazy can be attributed to its Rust-based execution engine and multi-threaded architecture which achieves efficient parallel processing. In contrast, Baseline Pandas has the longest execution time because it relies on sequential processing without optimization technology.



Figure 6.1 Execution Time Comparison

Memory Usage Comparison

Figure 6.2 illustrates the memory consumption of each framework during operation. The Optimized Pandas records the lowest memory utilization rate, showing the effectiveness of explicit data type specifications and column trimming in reducing memory overhead. DuckDB SQL and Polars Lazy Execution consume moderate memory, and the Baseline Pandas has the highest memory utilization rate due to its lack of memory optimization strategies.

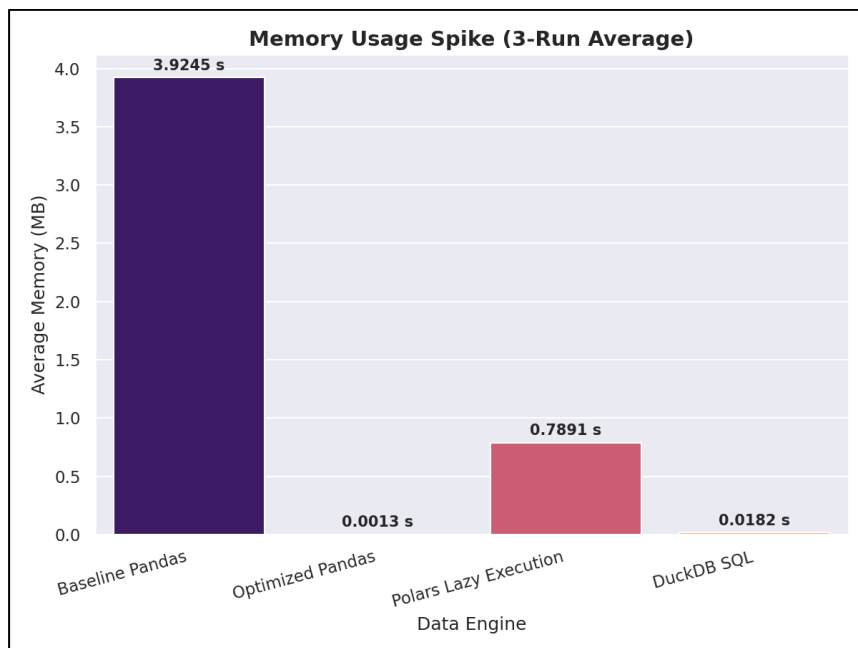


Figure 6.2 Memory Usage Comparison

CPU Usage Comparison

Figure 6.3 compares the CPU utilization rate of the four frameworks. Polars Lazy Execution has the lowest CPU utilization, followed by Baseline Pandas, DuckDB SQL and Optimized Pandas. Although Polars uses the least CPU resources, it still achieves the fastest execution time and the highest throughput, demonstrating excellent computing efficiency. The optimized Pandas shows the highest CPU utilization rate, which indicates that additional optimization operations increase the processor workload during execution.

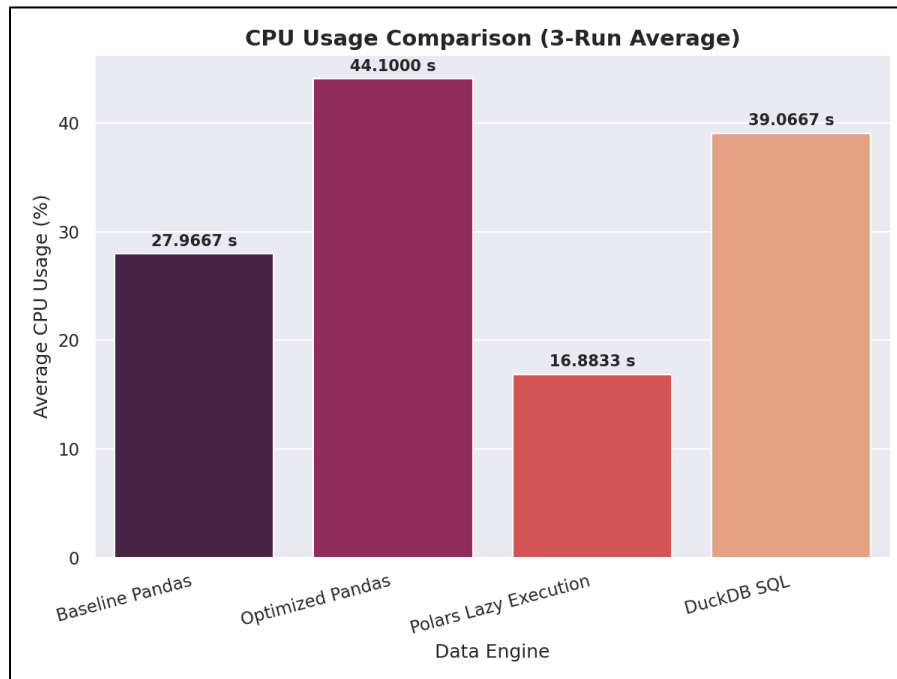


Figure 6.3 CPU Usage Comparison

Throughput Comparison

Figure 6.4 shows the throughput achieved by each framework. Polars Lazy Execution achieves the highest throughput, followed by Optimized Pandas, Baseline Pandas and DuckDB SQL. The results show that Polars Lazy can process more records per second than other frameworks which highlights the effectiveness of its parallel execution architecture.

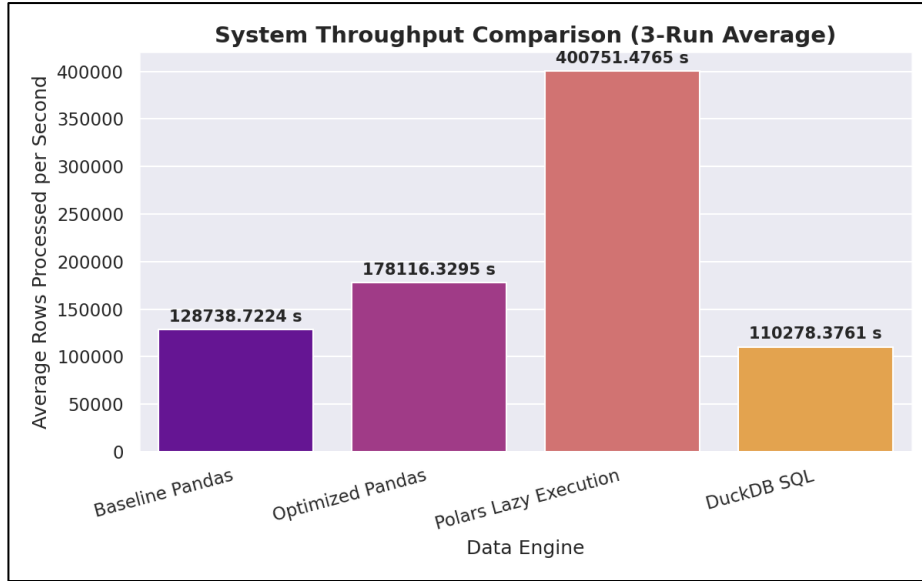


Figure 6.4 Throughput Comparison

Overall, the benchmark results that Polars Lazy Execution provides the best overall performance in the framework of evaluation. It achieves the shortest execution time, the highest throughput and the lowest CPU utilization, indicating efficient resources utilization and excellent processing power. The Optimized Pandas good in memory efficiency and achieves the lowest memory consumption. Although DuckDB SQL is designed to analyse workloads, its performance is limited by initialization overhead when processing relatively small JobStreet datasets. Baseline Pandas performed averagely in most evaluation indicators. Although it does not achieve the fastest execution time or the lowest memory consumption, it maintains relatively stable throughput and CPU utilization. This show that the standard Pandas implementation is still a practical solution for medium-size datasets, although additional optimization technology can further improve efficiency.

7.0 Challenges & Limitations

7.1 Challenges

During the benchmarking process, the performance difference between some frameworks was smaller than originally expected. Although Polars Lazy Execution has always achieved the best overall performance, DuckDB SQL and Baseline Pandas provide relatively stable and competitive results. Initially, it is expected that the optimized framework will be much better than the baseline implementation in all aspects. However, the results show that Baseline Panda is still a practical solution for processing medium-sized datasets, while DuckDB provides balanced performance rather than substantial advantages. In addition, due to the shared cloud environment of Google Colab, small changes in benchmark measurements have been observed, which may affect the allocation of resources during operation.

7.2 Limitations of the solution

One of the limitations of this project is the size of the dataset used for evaluation. The benchmarking process is carried out using the JobStreet dataset containing 14,105 records. Although this dataset is sufficient to demonstrate data processing optimization techniques, the results may be different when a larger dataset is used. Therefore, the results of this study should be interpreted in the context of medium-sized datasets, not in a large-scale big data environment.

In addition, several optimization techniques implemented in the project rely on predefined column names and explicit data types. If the structure of the data set changes significantly, additional preprocessing and code modification may be required before the pipeline is successfully executed. When processing highly dynamic data sets obtained from different sources, this dependence reduces the flexibility of the solution.

Finally, performance evaluation mainly focuses on data loading, grouping and aggregation operations. More advanced analytical tasks, such as complex connections, feature engineering, machine learning preprocessing and large-scale data conversion, are not included in the baseline. Therefore, the performance of the report may not fully represent the ability of the evaluation framework in all data analysis scenarios.

8.0 Conclusion & Future Work

8.1 Summary of findings

This project has successfully demonstrated the end-to-end process of large-scale web crawling, data cleaning, data transformation, optimization and performance evaluation for JobStreet Malaysia job listing data. The project was developed to show how high-performance data processing techniques can be applied to collect and process a large amount of online job market data in a more efficient and structured way. The web crawler was created using Python to automatically extract job listing information from JobStreet Malaysia. The crawler collected data across 30 job classification categories including details such as job title, company name, location, classification and salary information. As a result, a total of 105,094 raw job listing records were successfully collected and stored in JSON format.

After the data collection phase, the raw dataset was processed through a cleaning and transformation pipeline using Python and Pandas. The initial raw dataset contained duplicate records, missing values, inconsistent text formats, location variations and unstructured salary values. During the cleaning process, duplicate records were removed, required fields with missing values were filtered out and missing salary values were filled with “Not Disclosed”. Some more other cleaning and transformation process is taken to improve data consistency. After the full cleaning and transformation process, the final cleaned dataset contained 25,209 rows and 7 columns.

The performance evaluation objective was also achieved by benchmarking several data processing frameworks, namely Baseline Pandas, Optimized Pandas, Polars Lazy Execution and DuckDB SQL. These frameworks were compared using four main performance metrics,

which were execution time, memory usage, CPU usage, and throughput. Based on the results, Polars Lazy Execution showed the best overall performance. It achieved the shortest average execution time of 0.07 seconds and the highest average throughput of 400,751.48 rows per second. This shows that Polars is highly efficient for processing structured datasets because it supports lazy evaluation, query optimization and parallel execution.

Besides that, Optimized Pandas showed strong memory efficiency because the memory usage was reduced through selective column loading, data type specification, and category-based data compression. DuckDB SQL also showed stable performance, especially for SQL-based analytical processing. However, its performance was slightly limited in this project because the dataset used for benchmarking was medium-sized, and the initialization overhead affected the overall execution time. Baseline Pandas remained usable for medium-sized datasets, but it did not perform as well as the optimized approaches in terms of execution speed and throughput.

Overall, this project shows that the choice of data processing framework has a clear impact on performance. Through this project, practical experience was gained in web crawling, data cleaning, data transformation and performance benchmarking. The project also highlighted real-world data engineering challenges, such as duplicated data, missing values, inconsistent formatting, salary text extraction, and performance differences between processing frameworks.

8.2 Improvements

Although the project was successfully completed, several improvements could still be made in future work. First, a larger dataset could be used to test the scalability of the pipeline more accurately. The current benchmark was performed on a medium-sized dataset, so the results may be different when millions of records are processed. By increasing the dataset size, the performance differences between Pandas, Polars and DuckDB could be evaluated more clearly.

Secondly, the benchmarking process could be improved by running the experiment in a more stable computing environment instead of Google Colab. Since Google Colab is a shared cloud environment, CPU and memory availability may change during execution. This may affect the consistency of the benchmark results. A dedicated cloud server such as Amazon EC2 or Microsoft Azure Virtual Machines with fixed resources would allow more reliable performance measurements.

Third, the data storage format could be improved by replacing CSV with a more efficient format such as Parquet. Parquet uses column-based storage which can reduce file size and improve data loading and query performance. The cleaned data could also be stored in PostgreSQL or MongoDB to support structured querying, easier updates and better management of large datasets.

Lastly, the pipeline could be made more automated and scalable. Workflow tools such as Apache Airflow could be used to schedule crawling, cleaning and benchmarking tasks.

Incremental crawling could also be implemented to collect only new or updated job listings, reducing unnecessary repeated crawling.

9.0 References

- [1] Jobstreet. (2025). *Jobs in Malaysia - Jan 2025* | Jobstreet. [online] Available at: <https://my.jobstreet.com/jobs>.
- [2] Hasan, J. (2025). Benchmarking Analytical Query Performance.
- [3] codecentric AG. (2025). *DuckDB vs. Polars vs. Pandas: Benchmark & Comparison*. [online] Available at: <https://www.codecentric.de/en/knowledge-hub/blog/duckdb-vs-dataframe-libraries> [Accessed 13 Jun. 2026].
- [4] Pandas (2024). *pandas documentation — pandas 1.0.1 documentation*. [online] pandas.pydata.org. Available at: <https://pandas.pydata.org/docs/>.

10.0 Appendices

Screenshots of Output

```
[21:03:10] [CLS-1] Worker done -- 13,230 cards.
[21:03:11] [CLS-9] Worker done -- 9,750 cards.
[21:03:11] [CLS-15] Worker done -- 0 cards.
[21:03:12] [CLS-11] Worker done -- 3,552 cards.
[21:03:12] [CLS-14] pg 10 (+30). Cat: 300 | Total: 105,064 / 105,000
[21:03:13] [CLS-16] Worker done -- 0 cards.
[21:03:13] [CLS-13] Worker done -- 330 cards.
[21:03:13] [CLS-2] Worker done -- 13,289 cards.
[21:03:14] [CLS-17] Worker done -- 0 cards.
[21:03:15] [CLS-7] pg 444 (+30). Cat: 13,226 | Total: 105,094 / 105,000
[21:03:15] [CLS-14] Worker done -- 300 cards.
[21:03:16] [CLS-18] Worker done -- 0 cards.
[21:03:17] [CLS-7] Worker done -- 13,226 cards.
[21:03:17] [CLS-19] Worker done -- 0 cards.
[21:03:19] [CLS-20] Worker done -- 0 cards.
[21:03:21] [CLS-21] Worker done -- 0 cards.
[21:03:22] [CLS-22] Worker done -- 0 cards.
[21:03:24] [CLS-23] Worker done -- 0 cards.
[21:03:26] [CLS-24] Worker done -- 0 cards.
[21:03:27] [CLS-25] Worker done -- 0 cards.
[21:03:29] [CLS-26] Worker done -- 0 cards.
[21:03:31] [CLS-27] Worker done -- 0 cards.
[21:03:32] [CLS-28] Worker done -- 0 cards.
[21:03:34] [CLS-29] Worker done -- 0 cards.
[21:03:35] [CLS-30] Worker done -- 0 cards.
[21:03:36] =====
[21:03:36] DONE -- 105,094 cards in 64.3 min
[21:03:36] Classification 1: 13,230
[21:03:36] Classification 2: 13,289
[21:03:36] Classification 3: 3,176
[21:03:36] Classification 4: 8,831
[21:03:36] Classification 5: 12,786
[21:03:36] Classification 6: 12,893
[21:03:36] Classification 7: 13,226
[21:03:36] Classification 8: 9,554
[21:03:36] Classification 9: 9,750
[21:03:36] Classification 10: 3,873
[21:03:36] Classification 11: 3,552
[21:03:36] Classification 12: 304
[21:03:36] Classification 13: 330
[21:03:36] Classification 14: 300
[21:03:36] File: C:\Users\Polok\OneDrive - Universiti Teknologi Malaysia (UTM)\Documents\Y3S2\High performance Data Processing\Project\data\raw_data.json (19.6 MB)
```

Links to full code repo or dataset

Group GitHub: <https://github.com/sean-seah/HPDP/tree/main/2526/project/BigPotato>

Datasets: <https://github.com/sean-seah/HPDP/tree/main/2526/project/BigPotato/data>

Main Crawler: https://github.com/sean-seah/HPDP/blob/main/2526/project/BigPotato/p1/main_crawler.ipynb

Data Cleaning: https://github.com/sean-seah/HPDP/blob/main/2526/project/BigPotato/p1/clean_data.ipynb

Optimization: https://github.com/sean-seah/HPDP/blob/main/2526/project/BigPotato/p1/optimize_pipeline.ipynb